МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
**Федеральное государственное бюджетное образовательное учреждение высшего образования**
**«Кемеровский государственный университет»**
**Институт Цифры**

# ДОМАШНЯЯ РАБОТА №1

# ПО ДИСЦИПЛИНЕ

# “Параллельные вычисления”

**Задача № ____**

студентов 3 курса

**Сизов Максим Евгеньевич**

**Грачев Ярослав Евгеньевич**

Направление 02.03.02 – Фундаментальная информатика и информационные технологии

Преподаватель:
к-т физ.-мат.наук, доцент
С.В. Стуколов

_____

Работа защищена:
“____”_____2026г.

с оценкой _____

Кемерово 2026

СОДЕРЖАНИЕ

1. ПОСТАНОВКА ЗАДАЧИ ..... 2
2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ФУНКЦИЙ ..... 3
3. ОПИСАНИЕ ПРОГРАММЫ ..... 4
4. РЕАЛИЗАЦИЯ ..... 5
ЗАКЛЮЧЕНИЕ ..... 9
ЛИТЕРАТУРА ..... 10

## 1. Постановка задачи

Цель работы: изучение базовых возможностей библиотеки MPI для создания параллельных программ; освоение функций инициализации, завершения, идентификации процессов, получения информации об узлах и измерения времени; получение практических навыков компиляции и запуска программ с MPI.

Задачи:
1. Скачайте, установите и настройте библиотеку mpich на домашнем компьютере. В файле с описанием выполнения задания приведите скрины, демонстрирующие шаги подключения библиотеки, компиляции и запуска 24 тестовой MPI-программы. Укажите характеристики компьютера, операционную систему, используемое программное обеспечение.

2. Дайте описание функции MPI_Get_processor_name и приведите пример ее использования.

3. Дайте описание и продемонстрируйте выполнение функции MPI_Wtick().

4. Напишите последовательную программу суммирования ряда чисел (1/(1+i)). Проведите исследование времени выполнения программы в зависимости от предела ряда. Предел ряда взять 10^6 , 10^7 … 10^9. Для определения времени выполнения программы используйте функцию MPI_Wtime(), описание которой также приведите в отчете. Полученные данные приведите в табличном виде (предел ряда, результат суммирования, время выполнения).

5. Напишите программу, которая выводит на каждом процессе номер процесса (rank) и переменную n, которая вычисляется следующим образом: на четных – n=2*rank, на нечетных – n=3*rank. Запуск параллельной программы осуществите на различном количестве процессов (например, size=10,15,20).

6. Напишите программу, которая определяет среднее арифметическое 100 чисел, заданных случайным образом от 0 до 100 на каждом процессе. Вывод должен содержать номер процесса и вычисленное среднее значение случайных чисел для данного процесса.

7. Напишите программу, в которой на каждом процессе создается одномерный массив a[n], инициализация элементов которого происходит на каждом процессоре одинаково a[i]=i. Затем, каждый процесспроизводит вычисление сумм первых k элементов данного массива, где k – номер процесса, увеличенный на 1. Сделайте контрольный вывод результатов: номер процессора, результат вычислений.

## 2. Описание используемых функций

2.1 MPI_Init
Назначение: Инициализация библиотеки MPI. Должна вызываться первой в программе.
Входные параметры: argc, argv - параметры командной строки (передаются из main)
Выходные параметры: код ошибки

2.2 MPI_Finalize
Назначение: Завершение работы библиотеки MPI. Освобождает все ресурсы.

2.3 MPI_Comm_size
Назначение: Определение количества процессов в области связи.
Входные параметры: comm - коммуникатор (обычно MPI_COMM_WORLD)
Выходные параметры: size - количество процессов

2.4 MPI_Comm_rank
Назначение: Определение индивидуального номера (ранга) текущего процесса.
Синтаксис (C):
Входные параметры: comm - коммуникатор
Выходные параметры: rank - номер процесса (от 0 до size-1)

2.5 MPI_Get_processor_name
Назначение: Получение имени узла (хоста), на котором выполняется процесс.
Входные параметры: name - буфер для имени (максимум MPI_MAX_PROCESSOR_NAME)
Выходные параметры: resultlen - длина имени

2.6 MPI_Wtime
Назначение: Возвращает текущее астрономическое время в секундах.
Возвращаемое значение: текущее время в секундах (тип double)

2.7 MPI_Wtick
Назначение: Возвращает разрешение (точность) системного таймера.
Возвращаемое значение: минимальный квант времени в секундах

## 3. Описание программы

1 - вывод ранга каждого процесса и общего количества процессов

2 - программа получает имя узла через MPI_Get_processor_name и выводит его. Поскольку все процессы запускаются на одной машине, имена узлов совпадают

3 - вывод разрешения таймера, возвращаемого функцией MPI_Wtick

4 - вычисление суммы ряда $S = \Sigma\ 1/(1+i)$ для значений предела $10^6 \dots 10^9$. Время выполнения замеряется через MPI_Wtime(). Для получения единой таблицы вычисления выполняются на процессе с рангом 0.

5 - использование ранга процесса для ветвления: на чётных процессах n = 2*rank, на нечётных - n = 3*rank.

6 - генерация 100 случайных чисел от 0 до 100 на каждом процессе и вычисление среднего. Разные последовательности обеспечиваются использованием srand(time(NULL) + rank).

7 - создаётся массив a[i] = i на каждом процессе, затем процесс суммирует первые k = rank + 1 элементов массива. Ограничение i < len защищает от выхода за границы массива, если процессов больше, чем элементов.

# 4. Реализация

1)

```
→ lab1 inst openmpi-bin openmpi-common libopenmpi-dev
[sudo] пароль для yaroslav:
Сущ:1 http://archive.ubuntu.com/ubuntu noble InRelease
Сущ:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Сущ:3 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Пол:4 https://repo.steampowered.com/steam stable InRelease [3 622 B]
Сущ:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Сущ:6 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu noble InRelease
Сущ:7 https://ppa.launchpadcontent.net/ubuntu-toolchain-r/test/ubuntu noble InRelease
Сущ:8 https://packages.mozilla.org/apt mozilla InRelease
Получено 3 622 B за 12с (308 B/s)
Чтение списков пакетов… Готово
Построение дерева зависимостей… Готово
Чтение информации о состоянии… Готово
Может быть обновлён 31 пакет. Запустите «apt list --upgradable» для показа.
Чтение списков пакетов… Готово
Построение дерева зависимостей… Готово
Чтение информации о состоянии… Готово
Уже установлен пакет openmpi-bin самой новой версии (4.1.6-7ubuntu2).
Следующие пакеты устанавливались автоматически и больше не требуются:
  dkms gcc-14-base:i386 hwloc-nox libavcodec60:i386 libavutil58:i386 libcodec2-1.2:i386 libdav1d7:i386 libfwupd2 libgomp1:i386
  libgsm1:i386 libmpich12 libnuma1:i386 libopenjp2-7:i386 libshine3:i386 libslurm40t64 libsnappy1v5:i386 libsoxr0:i386
  libspeexdsp1:i386 libsvtav1enc1d1:i386 libswresample4:i386 libvdpau1:i386 libwebpmux3:i386 libx264-164:i386 libx265-199:i386
  libxvidcore4:i386 libzvbi0t64:i386 mesa-vdpau-drivers:i386 nvidia-firmware-535-535.288.01 nvidia-firmware-535-535.309.01
  nvidia-firmware-580-580.173.02 nvidia-firmware-595-595.58.03 vdpau-driver-all:i386
Для их удаления используйте «sudo apt autoremove».
Обновлено 0 пакетов, установлено 0 новых пакетов, для удаления отмечено 0 пакетов, и 31 пакетов не обновлено.
```

*Ставил OpenMPI вместо MPICH, т. к. версия последнего на репозиториях Ubuntu работает некорректно, отказываясь запускать программы параллельно. С точки зрения работы и синтаксиса они не различаются.

```
#include <mpi.h>
#include <stdio.h>


int main(int argc, char** argv) {
	MPI_Init(&argc, &argv);
	int rank, size;
	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);

	printf("rank=%d, size=%d\n",  rank, size);

	MPI_Finalize();
	return 0;
}
```

```
→  lab1 mpiexec -n 4 first
rank=0, size=4
rank=1, size=4
rank=2, size=4
rank=3, size=4
```

2)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
	MPI_Init(&argc, &argv);

	int rank, size, len;

	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);

	  char host[MPI_MAX_PROCESSOR_NAME];
	  MPI_Get_processor_name(host, &len);

	printf("Processor name: %s\n", host);

	MPI_Finalize();
	return 0;
}
```

```
→  lab1 mpiexec -n 4 second
Processor name: yaroslav-MS-TZZ-H311M-VH-M-2
Processor name: yaroslav-MS-TZZ-H311M-VH-M-2
Processor name: yaroslav-MS-TZZ-H311M-VH-M-2
Processor name: yaroslav-MS-TZZ-H311M-VH-M-2
```

3)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
   MPI_Init(&argc, &argv);

   int rank, size;
   double time = MPI_Wtick();

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   printf("Tick: %e\n", time);

   MPI_Finalize();
```

```
    return 0;
}
```

```
➜  lab1 mpiexec -n 4 thirth
Tick: 1.000000e-09
Tick: 1.000000e-09
Tick: 1.000000e-09
Tick: 1.000000e-09
```

4)

```
#include <mpi.h>
#include <stdio.h>
#include <math.h>

int main(int argc, char** argv) {
	MPI_Init(&argc, &argv);
	int rank, size;
	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);

	double start = MPI_Wtime();

	double sum = 0;
	double limit = pow(10, rank + 6);

	for(int i = 0; i < limit; i++)
		sum += (1.0 / (1 + i));

	double end = MPI_Wtime();

	printf("Limit: %e | Result: %e | Time: %e\n", limit, sum, end - start);

	MPI_Finalize();
	return 0;
}
```

```
➜  lab1 mpiexec -n 4 fourth
Limit: 1.000000e+06 | Result: 1.439273e+01 | Time: 5.028722e-03
Limit: 1.000000e+07 | Result: 1.669531e+01 | Time: 3.165647e-02
Limit: 1.000000e+08 | Result: 1.899790e+01 | Time: 2.403398e-01
Limit: 1.000000e+09 | Result: 2.130048e+01 | Time: 2.322178e+00
```

5)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size, n;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    if (rank % 2 == 0) n = 2 * rank;
    else n = 3 * rank;

    printf("Rank: %2d | n: %2d\n", rank, n);

    MPI_Finalize();
    return 0;
}
```

```
→ lab1 mpiexec -n 4 fifth
Rank:  0 | n:  0
Rank:  1 | n:  3
Rank:  2 | n:  4
Rank:  3 | n:  9
```

6)

```
#include <mpi.h>
#include <stdio.h>
#include <time.h>
#include <stdlib.h>

int main(int argc, char** argv) {
        MPI_Init(&argc, &argv);

        int rank, size, sum = 0;

        MPI_Comm_rank(MPI_COMM_WORLD, &rank);
        MPI_Comm_size(MPI_COMM_WORLD, &size);

           srand(time(NULL) + rank);

           for (int i = 0; i < 100; i++) {
                   sum += rand() % 100;
        }

           printf("Rank: %3d | Avg: %3f\n", rank, sum / 100.0);

        MPI_Finalize();
        return 0;
}
```

```
→ lab1 mpiexec -n 100 --oversubscribe sixth
Rank:  41 | Avg: 52.620000
Rank:  61 | Avg: 50.120000
Rank:  66 | Avg: 48.930000
Rank:  12 | Avg: 53.410000
Rank:  47 | Avg: 54.120000
Rank:  60 | Avg: 46.270000
Rank:  48 | Avg: 52.240000
Rank:  43 | Avg: 48.020000
Rank:  85 | Avg: 45.860000
Rank:  73 | Avg: 54.110000
Rank:   1 | Avg: 51.230000
Rank:  87 | Avg: 53.300000
Rank:  95 | Avg: 48.930000
Rank:  98 | Avg: 48.950000
Rank:  99 | Avg: 47.830000
Rank:   0 | Avg: 49.550000
Rank:   2 | Avg: 52.700000
Rank:   4 | Avg: 52.860000
Rank:   5 | Avg: 48.490000
Rank:   6 | Avg: 49.030000
Rank:   8 | Avg: 46.250000
Rank:   9 | Avg: 51.100000
Rank:  14 | Avg: 49.620000
Rank:  15 | Avg: 45.170000
Rank:  17 | Avg: 55.780000
Rank:  18 | Avg: 52.680000
Rank:  19 | Avg: 50.380000
Rank:  20 | Avg: 50.190000
Rank:  22 | Avg: 50.910000
Rank:  27 | Avg: 46.310000
Rank:  28 | Avg: 54.990000
Rank:  31 | Avg: 51.510000
Rank:  36 | Avg: 50.000000
Rank:  39 | Avg: 48.480000
Rank:  45 | Avg: 51.460000
Rank:  46 | Avg: 56.030000
Rank:  49 | Avg: 48.460000
Rank:  50 | Avg: 47.200000
Rank:  51 | Avg: 52.040000
Rank:  52 | Avg: 49.100000
Rank:  53 | Avg: 47.170000
Rank:  56 | Avg: 51.230000
Rank:  57 | Avg: 48.900000
Rank:  58 | Avg: 50.330000
Rank:  59 | Avg: 45.630000
Rank:  62 | Avg: 52.020000
Rank:  64 | Avg: 50.410000
Rank:  65 | Avg: 52.010000
Rank:  68 | Avg: 48.150000
Rank:  69 | Avg: 45.380000
Rank:  70 | Avg: 49.440000
Rank:  72 | Avg: 48.480000
Rank:  75 | Avg: 50.460000
Rank:  76 | Avg: 50.550000
Rank:  78 | Avg: 47.560000
Rank:  81 | Avg: 51.390000
Rank:  82 | Avg: 50.130000
Rank:  83 | Avg: 48.000000
Rank:  88 | Avg: 45.660000
Rank:  89 | Avg: 48.790000
Rank:  90 | Avg: 52.140000
Rank:  92 | Avg: 49.000000
Rank:  93 | Avg: 48.590000
Rank:  21 | Avg: 44.680000
Rank:  26 | Avg: 52.720000
Rank:  34 | Avg: 51.260000
Rank:  38 | Avg: 45.810000
Rank:  79 | Avg: 49.150000
Rank:   7 | Avg: 49.310000
Rank:  13 | Avg: 53.780000
Rank:  25 | Avg: 57.830000
Rank:  33 | Avg: 48.230000
Rank:  37 | Avg: 51.480000
Rank:  44 | Avg: 50.230000
Rank:  55 | Avg: 54.590000
Rank:  71 | Avg: 51.310000
Rank:  94 | Avg: 51.540000
Rank:   3 | Avg: 49.050000
Rank:  10 | Avg: 48.500000
Rank:  74 | Avg: 49.840000
Rank:  11 | Avg: 47.610000
Rank:  84 | Avg: 45.910000
Rank:  86 | Avg: 51.100000
Rank:  96 | Avg: 44.490000
Rank:  16 | Avg: 50.910000
Rank:  24 | Avg: 46.560000
Rank:  30 | Avg: 52.510000
Rank:  32 | Avg: 54.330000
Rank:  42 | Avg: 48.910000
Rank:  54 | Avg: 45.490000
Rank:  63 | Avg: 51.390000
Rank:  67 | Avg: 47.730000
Rank:  35 | Avg: 49.820000
Rank:  40 | Avg: 52.680000
Rank:  91 | Avg: 47.540000
Rank:  80 | Avg: 50.860000
Rank:  23 | Avg: 45.020000
Rank:  97 | Avg: 44.390000
Rank:  29 | Avg: 53.710000
Rank:  77 | Avg: 50.730000
```

7)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
	MPI_Init(&argc, &argv);

	int rank, size, len = 10, sum = 0;

	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);

	   int a[len];
	   for (int i = 0; i < len; i++) a[i] = i;
	   for (int k = 0; k <= rank; k++) sum += k;

	   printf("Rank: %d | Result: %d\n", rank, sum);

	MPI_Finalize();
	return 0;
}
```

```
→  lab1 mpiexec seventh
Rank: 3 | Result: 6
Rank: 4 | Result: 10
Rank: 5 | Result: 15
Rank: 0 | Result: 0
Rank: 1 | Result: 1
Rank: 2 | Result: 3
```

## Заключение

В ходе лабораторной работы были изучены и практически освоены базовые функции библиотеки MPI:

Обрамляющие функции MPI_Init и MPI_Finalize

Функции идентификации MPI_Comm_size и MPI_Comm_rank

Функция MPI_Get_processor_name

Таймеры MPI_Wtime и MPI_Wtick

Приобретённые навыки:

1.Установка и настройка MPICH, компиляция (mpicc) и запуск (mpiexec) MPI-программ.

2.Использование ранга процесса для ветвления алгоритма.

3.Измерение времени выполнения и оценка вычислительной сложности (линейный рост времени с ростом предела ряда).

4.Генерация независимых последовательностей случайных чисел на разных процессах.

5.Работа с массивами в параллельной программе.

6.Понимание того, что MPI позволяет запускать несколько процессов даже на одном узле, при этом имена узлов совпадают.

Литература

1.Стуколов С.В. Параллельное программирование. Практикум